



Gitra Platform

Internal Git & CI/CD Platform on Azure — Project Documentation

Member	Workstream
Nasser	Azure Infrastructure & Terraform
Faisal	GitLab & CI/CD Configuration
Mohammed	Docker, Bash Automation & Operations

Live environment: <https://gitra-25ee91e6.eastus.cloudapp.azure.com>

Repository: github.com/Mohammed-Alghumayti/gitra-platform (branch Testing) · 2026-10-07

Contents

Part 1 — Architecture

- What this covers
- System overview
- Member 1 — Azure Infrastructure & Terraform
- Member 2 — GitLab & CI/CD
- Member 3 — Docker, Bash Automation & Operations
- Network
- Security model

Part 2 — Deployment & Operations

- What this covers
- One-time setup (~10 minutes, all in the browser)
- What every deployment does
- After the first deployment
- Starting and stopping the servers
- Azure resource groups — what's safe to delete
- Backup and restore
- Server access (admin SSH)
- Manual install (without GitHub Actions)
- Local checks (no Azure needed)

Part 3 — Troubleshooting & Change Log

- What this covers
- Change log
- Troubleshooting guide
- Still worth improving

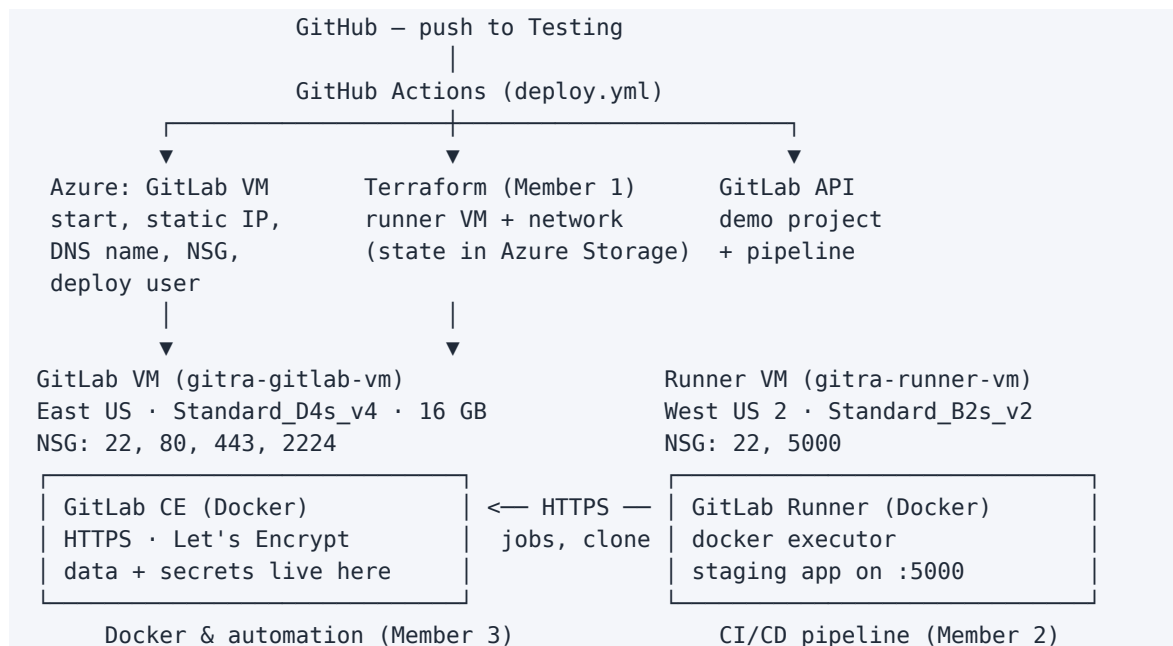
Part 1 — Architecture

What this covers

Gitra is an internal Git and CI/CD platform built on **GitLab CE**, running on **Microsoft Azure**. It gives the team one place for source control, code review (merge requests) and CI/CD pipelines. It is publicly reachable so supervisors and other students can use it, with security at every layer, and it is deployed **fully automatically** by GitHub Actions.

Member	Workstream	Main folders
Member 1 — Nasser	Azure Infrastructure & Terraform	<code>terraform/gitlab</code> , <code>terraform/runner</code>
Member 2 — Faisal	GitLab & CI/CD Configuration	<code>sample-app/</code> , <code>runner</code>
Member 3 — Mohammed	Docker, Bash Automation & Operations	<code>docker-compose*.yaml</code> , <code>scripts/</code> , <code>.github/</code>

System overview



Key design decisions

Decision	Why
GitLab and the runner on separate VMs	The runner mounts the Docker socket so jobs can build images, which gives jobs root-level control of their VM. Keeping it away from GitLab means a malicious <code>.gitlab-ci.yml</code> can't reach GitLab's data or secrets.
Runner VM in another region	The subscription allows 4 vCPUs per region ; the GitLab VM uses all 4 in East US. The runner talks to GitLab only over HTTPS, so distance doesn't matter.
Public, but hardened	Supervisors and students need access, so IP allow-lists aren't an option. Instead: HTTPS, admin-approved sign-up, mandatory 2FA, private code.
Everything in Docker	Isolated from the host OS, easy to upgrade (change the image), identical everywhere.

Decision	Why
Infrastructure as Code	Terraform defines the runner VM; the workflow can rebuild it at any time and changes are reviewed in Git.
Idempotent automation	Every deployment step is safe to re-run, so a failed run can simply be run again.

Why two VMs instead of single-node

The project spec proposes a *single-node internal deployment model* — GitLab and its runner on one VM — for simplicity. We kept everything else in the spec but split the runner onto its own VM:

	Single-node (spec)	Two VMs (what we built)
Security	The runner mounts the Docker socket, so every CI job has root-level control of the machine — on a single node that is GitLab's machine, with its database, repositories and <code>gitlab-secrets.json</code>	CI jobs can only reach the runner VM; GitLab's data and secrets aren't there
Performance	<code>docker build</code> and tests compete with GitLab (which needs ~4 GB RAM on its own)	GitLab keeps its whole VM
Azure quota	GitLab already uses the subscription's 4-vCPU quota in East US; a bigger single VM wouldn't fit	The runner uses another region's quota
Recovery	A runaway job can take GitLab down	The runner VM is disposable — Terraform rebuilds it on the next run
Cost	One VM	One extra small VM (~\$30–40/month; can be stopped)
Complexity	Simpler	Automated away: the same workflow builds both

The spec itself lists the CI runner as *optional configuration*; we placed it where it can't hurt the platform. This follows GitLab's own guidance to not run runners on the GitLab server, and it was the first security risk raised in the design (see *Security model*).

Member 1 — Azure Infrastructure & Terraform

What this covers

Terraform defines all the Azure infrastructure, in two configurations with separate state files — one for the **GitLab platform**, one for the **CI runner** — so a change to one can never touch the other. Each configuration is a single `main.tf`, split into clearly marked sections (settings, variables, resource group, network, security, VM, outputs); Terraform reads all `.tf` files in a folder as one, so file layout doesn't change what gets built.

Repository layout

```

terraform/
├── gitlab/
│   └── main.tf
│   |
│   |
│   |
│   |
│   |
│   |
│   |
│   |
│   |
│   |
└──

```

```

# GitLab platform (state: gitra-gitlab.tfstate)
# Everything, in sections:
#   settings: azurerm + random, remote state in Azure Storage
#   variables: region, zone, VM size, DNS label, address ranges
#   resource group gitra-rg
#   network: VNet, subnet, static public IP + DNS name, NIC
#   security: NSG (22, 80, 443, 2224) + NIC association
#   VM: gitra-gitlab-vm (Standard_D4s_v4, Trusted Launch)
#   outputs: GitLab URL, public IP

```

```

|   └─ import_existing.sh      # Adopts the existing resources into the state
|   └─ tests/platform.tftest.hcl # Names, sizes, ranges, NSG ports
└─ runner/
    └─ main.tf                 # CI runner (state: gitra-runner.tfstate)
        # Everything, in sections:
        #   settings, variables, resource group gitra-runner-rg
        #   network: VNet, subnet, static public IP, NIC
        #   security: NSG (22, 5000) + NIC association
        #   VM: gitra-runner-vm, SSH keys only
        #   outputs: runner IP, SSH command, staging URL
    └─ cloud-init-runner.yaml.tftpl # First boot: Docker, fail2ban, clone repo, start runner
    └─ tests/plan.tftest.hcl      # Runner config, SSH keys only, NSG ports, names

```

Resources

Config	Resource	Name	Notes
gitlab	Resource group	<code>gitra-rg</code>	<code>prevent_destroy</code>
gitlab	VNet / subnet	<code>vnet-eastus-1 / snet-eastus-1</code>	<code>172.16.0.0/16 / 172.16.0.0/24</code>
gitlab	Public IP	<code>gitra-gitlab-vm-ip</code>	Static, zone 1, DNS <code>gitra-25ee91e6</code> — <code>prevent_destroy</code>
gitlab	NSG	<code>gitra-gitlab-vm-nsg</code>	22, 80, 443, 2224
gitlab	VM	<code>gitra-gitlab-vm</code>	<code>Standard_D4s_v4</code> (4 vCPU, 16 GB), zone 1, Trusted Launch, 30 GB Premium SSD — <code>prevent_destroy</code>
runner	Resource group	<code>gitra-runner-rg</code>	Runner only
runner	VNet / subnet	<code>gitra-runner-vnet / gitra-runner-subnet</code>	<code>10.20.0.0/16 / 10.20.1.0/24</code>
runner	Public IP	<code>gitra-runner-pip</code>	Static
runner	NSG	<code>gitra-runner-nsg</code>	22, 5000
runner	VM	<code>gitra-runner-vm</code>	First 2-vCPU size with capacity (currently <code>Standard_B2s_v2</code>), 32 GB Premium SSD
both	State storage	<code>gitra-tfstate-rg / gitratf...</code>	Created by the workflow

Notable implementation details

- **Adopting the existing platform.** The GitLab VM was created before Terraform managed it. `import_existing.sh` imports each resource into the state (importing changes nothing in Azure); the configuration was written from the VM's actual Azure settings so the plan matches reality.
- **Data safety.** `prevent_destroy` on the resource group, public IP and VM. The workflow refuses to apply a plan that deletes or replaces anything, and doesn't auto-apply a plan that would modify an existing resource.
- **Settings that would force a rebuild are ignored** (`ignore_changes` on the admin password, SSH key, custom data and image version), so Terraform never recreates the VM over them.
- **Remote state** in Azure Storage, so every GitHub Actions run sees the same infrastructure.
- **Runner:** `replace_triggered_by` rebuilds the subnet with its VNet and the NIC–NSG link with the NIC; `prevent_deletion_if_contains_resources = false` lets `gitra-runner-rg` move region even if a failed attempt left a disk.
- **Tests without Azure:** `terraform test` with mocked providers in both configurations.

Member 2 — GitLab & CI/CD

What this covers

The GitLab runner that executes pipelines, a demo application, and its CI/CD pipeline.

Repository layout

```

docker-compose.runner.yaml    # GitLab Runner container (runner VM)
sample-app/
├─ app.py                    # Flask app: / and /health
├─ test_app.py               # pytest tests
├─ requirements.txt
├─ Dockerfile                # python:3.12-slim, runs as a non-root user
├─ .gitlab-ci.yml            # test → build → deploy_staging
└─ README.md

```

Runner

- **gitlab-runner** container on the runner VM, **Docker executor** (every job runs in a fresh container).
- Registered automatically by the workflow; talks to GitLab over HTTPS.

Pipeline

Stage	Job	What it does
test	test	pip install + pytest (image python:3.12)
build	build	docker build , tagged with the commit SHA and latest
deploy	deploy_staging	Replaces the internal-demo-app-staging container on port 5000 and checks /health (default branch only)

If a stage fails, the next ones don't run — broken code never reaches staging.

Member 3 — Docker, Bash Automation & Operations

What this covers

Running GitLab in Docker with persistent storage, the scripts to deploy, health-check, harden and back it up, and the GitHub Actions workflow that automates everything.

Repository layout

```

docker-compose.yaml          # GitLab CE (GitLab VM)
.env.example                  # GITLAB_EXTERNAL_URL (the workflow writes .env)
scripts/
├─ deploy.sh                  # Storage folders + launch GitLab
├─ health_check.sh            # Waits until GitLab is really ready (/--/readiness + HTTP)
├─ harden_gitlab.sh           # Sign-up approval, 2FA, password policy, no public projects
├─ deploy_runner.sh           # Launch the runner (runner VM)
├─ register_runner.sh         # Register the runner with GitLab
├─ backup.sh                  # Backup + gitlab-secrets.json + gitlab.rb
├─ setup.sh                   # Installs Docker (manual path)
└─ ci/                        # Used by the workflow
    ├─ azure_prepare_gitlab_vm.sh # Start VM, static IP, DNS name, NSG, deploy user
    ├─ remote_deploy_gitlab.sh    # Deploy GitLab, keep data, repair permissions
    ├─ gitlab_bootstrap.rb        # Root password, runner token, temporary token
    └─ run_gitlab_bootstrap.sh    # Runs it over SSH (secrets via stdin, retries)

```

```
└─ push_demo_app.sh          # Push sample-app/ and wait for its pipeline
.github/workflows/deploy.yml # The automatic deployment
```

Persistent storage

VM	Host path	Container path	Contents
GitLab	<code>/opt/gitra-platform/gitlab/config</code>	<code>/etc/gitlab</code>	Configuration, secrets, certificates
GitLab	<code>/opt/gitra-platform/gitlab/logs</code>	<code>/var/log/gitlab</code>	Logs
GitLab	<code>/opt/gitra-platform/gitlab/data</code>	<code>/var/opt/gitlab</code>	Database, repositories, uploads
Runner	<code>~/gitra-platform/runner</code>	<code>/etc/gitlab-runner</code>	Runner config + token

The parent `./gitlab` folder is `root:700`. The mounted subfolders are managed by GitLab itself — its internal users (`git`, `gitlab-www`, `gitlab-psql`...) need their own permissions there.

Network

VM	Rule	Port	Purpose
GitLab	Allow-SSH-Admin	22	Server administration (keys only + fail2ban)
GitLab	Allow-HTTP	80	Redirect to HTTPS + Let's Encrypt validation
GitLab	Allow-HTTPS	443	GitLab web interface
GitLab	Allow-GitLab-SSH	2224	Git clone / push / pull over SSH
Runner	Allow-SSH-Admin	22	Server administration (keys only + fail2ban)
Runner	Allow-Staging-App	5000	Demo app deployed by the pipeline

Git SSH uses **2224** so it never conflicts with the VM's own admin SSH on 22, and each can be restricted separately.

Security model

Layer	Protection	Where
Encryption	HTTPS with a free Let's Encrypt certificate on a free Azure DNS name; HTTP redirects to HTTPS	<code>azure_prepare_gitlab_vm.sh</code> , <code>docker-compose.yaml</code>
Accounts	Anyone can register, but an admin must approve each account	<code>hardening_gitlab.sh</code>
Login	2FA mandatory (48h grace period); passwords ≥ 12 characters	<code>hardening_gitlab.sh</code>
Code	Projects can't be public — visible to signed-in users only	<code>hardening_gitlab.sh</code>
Brute force	GitLab's login rate limiting; fail2ban on SSH	GitLab, deploy scripts
Servers	Admin SSH accepts keys only (password and root login off); deploy user's key added through Azure	<code>hardening_ssh.sh</code> , <code>azure_prepare_gitlab_vm.sh</code> , <code>terraform/runner/main.tf</code>
Recovery	Backups include the secrets needed to decrypt them; <code>restore.sh</code> checks versions and secrets	<code>backup.sh</code> , <code>restore.sh</code>
Network	Each VM opens only the ports it needs	NSGs

Layer	Protection	Where
Isolation	CI jobs run on a separate VM from GitLab's data	Architecture
Files	Parent data folder <code>root:700</code> ; GitLab manages the rest	<code>deploy.sh</code>
Git	<code>.gitignore</code> excludes <code>gitlab/</code> , <code>runner/</code> , <code>.env</code> , backups, Terraform state, <code>*.tfvars</code>	<code>.gitignore</code>
Automation secrets	GitHub Secrets; masked in logs; passed via stdin / root-only env files, never on a command line; temporary GitLab token revoked every run	<code>deploy.yml</code> , <code>run_gitlab_bootstrap.sh</code>
Automation identity	Service principal with Contributor on the subscription (can't grant permissions)	Azure
Containers	The demo app runs as a non-root user	<code>sample-app/Dockerfile</code>

Part 2 — Deployment & Operations

What this covers

How the platform is deployed (automatically, with GitHub Actions), the one-time setup it needs, and day-to-day operations: logging in, inviting people, admin SSH access, starting and stopping the servers, backup and restore, and which Azure resources are safe to delete.

Live environment: <https://gitra-25ee91e6.eastus.cloudapp.azure.com>

One-time setup (~10 minutes, all in the browser)

1. Azure credentials and SSH key

Open [Azure Cloud Shell](#) (Bash) and run:

```
SUB=$(az account show --query id -o tsv)
az ad sp create-for-rbac --name gitra-github-actions --role Contributor \
  --scopes /subscriptions/$SUB --json-auth
ssh-keygen -t rsa -b 4096 -N "" -C gitra-deploy -f ~/gitra_deploy
cat ~/gitra_deploy
```

The first command prints a JSON block (the service principal GitHub Actions uses). The last prints a private SSH key. Keep both for step 2.

If `create-for-rbac` fails with an authorization error, your account isn't Owner on the subscription — ask the subscription owner to run it.

2. GitHub secrets

Repository **Settings** → **Secrets and variables** → **Actions** → **New repository secret**:

Secret	Value
<code>AZURE_CREDENTIALS</code>	The whole JSON block from step 1
<code>SSH_PRIVATE_KEY</code>	The whole private key (<code>-----BEGIN ...</code> to <code>-----END ...</code>)
<code>GITLAB_ROOT_PASSWORD</code>	A strong password for GitLab's <code>root</code> (12+ characters, not a common word)

3. Optional variables

Same page, **Variables** tab:

Variable	Default	Use
<code>GITLAB_VM_IP</code>	20.55.88.3	Public IP of the GitLab VM
<code>DNS_LABEL</code>	gitra-<8 chars>	Azure DNS name → <a href="https://<label>.<region>.cloudapp.azure.com">https://<label>.<region>.cloudapp.azure.com
<code>RUNNER_LOCATION</code>	tries <code>westus2</code> , <code>centralus</code> , <code>eastus2</code> , <code>westus3</code> , <code>northeurope</code>	Pin the runner VM's region
<code>RUNNER_VM_SIZE</code>	tries 7 small 2-vCPU sizes	Pin the runner VM's size

4. Run

Push to **Testing**, or open the latest run under **Actions** and click **Re-run all jobs**.

The **Run workflow** button only appears once the workflow file exists on the default branch (**main**). Until then, use a push or **Re-run**.

What every deployment does

`.github/workflows/deploy.yml` runs on every push to **Testing** (docs-only changes are skipped). Each step is idempotent — safe to re-run after a failure.

Step	What happens
0. GitLab platform	Terraform imports the existing platform into its state (first run only), then plans. It applies only if nothing would be deleted, replaced or modified; otherwise it stops with an error or a warning listing what differs
1. Prepare GitLab VM	Finds the VM by its IP, starts it if it's stopped , makes the IP static, adds the free DNS name, opens ports 22/80/443/2224 in its NSG if missing, adds the gitra-deploy user with the SSH key (through the Azure VM agent — no password needed)
2. Deploy GitLab	Installs Docker, git and fail2ban if needed, pulls the branch to <code>/opt/gitra-platform</code> , moves any older GitLab data there (nothing is lost), starts GitLab with HTTPS, repairs file permissions if GitLab can't read its own files, waits until GitLab is really ready, applies the security settings, and turns off password login for admin SSH (keys only)
2b. Root password	Sets root 's password from the <code>GITLAB_ROOT_PASSWORD</code> secret
3. Runner VM	Creates the Terraform state storage if needed, then Terraform creates/updates the runner VM. An existing VM keeps its size and region (it's never resized); a new one tries sizes and regions until Azure has capacity, cleaning up after each failed attempt. Starts the runner VM if it's stopped — if Azure has no capacity to start it, another size or region is used
4. Register runner	Creates a runner token in GitLab and registers the runner (only if it isn't registered yet)
5. Demo app	Creates <code>root/internal-demo-app</code> in GitLab, pushes <code>sample-app/</code> , and waits for its pipeline: test → build → deploy_staging
Clean-up	Revokes the temporary GitLab token used in step 5

The run's **Summary** page shows the GitLab URL, the demo project and the staging app URL.

A full first run takes about **20–30 minutes**; later runs are faster.

After the first deployment

1. Open the GitLab URL and log in as **root** with the `GITLAB_ROOT_PASSWORD` secret.
2. Set up **2FA** when asked (Google Authenticator or Microsoft Authenticator).
3. Check the runner: **Admin** → **CI/CD** → **Runners** → **gitra-runner** with a green dot.
4. Check the pipeline: `root/internal-demo-app` → **Build** → **Pipelines**.

Inviting the supervisor and students

1. Send them the GitLab URL — they click **Register**.
2. Approve them: **Admin** → **Users** → **Pending approval** → **Approve**.
3. On first login they must set up 2FA.

Changing root's password

Change the `GITLAB_ROOT_PASSWORD` secret; the next run applies it.

Starting and stopping the servers

VM	Resource group	Purpose
gitra-gitlab-vm	gitra-rg	GitLab (East US)
gitra-runner-vm	gitra-runner-rg	Runner + staging app (West US 2)

Stop (to save money)

Azure portal → **Virtual machines** → select the VM → **Stop** → wait for **Stopped (deallocated)**.

*Always stop from the portal. Shutting down from inside the VM leaves it **Stopped** but still billed.*

While stopped, data is kept and only disk and IP costs remain.

Start

- **Portal:** select the VM → **Start**. GitLab and the runner start on their own (**restart: always**); GitLab needs 3–5 minutes.
- **GitHub Actions: Re-run all jobs** — this starts **both VMs** automatically.

Auto-shutdown

If **Auto-shutdown** is enabled on a VM (VM → **Operations** → **Auto-shutdown**), it stops every day at the set time. The next deployment starts the GitLab VM again.

Azure resource groups — what's safe to delete

Resource group	Contains	Delete?
gitra-rg	GitLab VM and all its data (projects, users, code)	❌ Never — this can't be undone
gitra-tfstate-rg	Terraform's state file	❌ No — costs almost nothing; without it Terraform forgets what it built
gitra-runner-rg	Runner VM only	⚠️ Safe — pipelines stop until the next deployment recreates it

To save money, **stop** VMs instead of deleting anything.

Backup and restore

Backup

On the GitLab VM:

```
sudo /opt/gitra-platform/scripts/backup.sh /root/gitlab-backups
```

It creates a GitLab backup (`_gitlab_backup.tar` — projects, users, issues, CI history) and copies it to the folder together with `gitlab-secrets.json` and `gitlab.rb` — **both are required to restore**. Treat the folder as sensitive and keep a copy off the VM. Backups older than 7 days are removed from the container.

Restore

⚠️ A restore **replaces all current GitLab data with the backup**.

```
sudo /opt/gitra-platform/scripts/restore.sh /root/gitlab-backups # newest backup in the folder
```

```
sudo /opt/gitra-platform/scripts/restore.sh /root/gitlab-backups <backup_id> # a specific one
```

It asks you to type **yes**, then:

#	Step	Why
1	Checks the backup's GitLab version (the end of <code><backup_id></code>) matches the running GitLab	GitLab can only restore a backup made by the same version
2	Restores <code>gitlab-secrets.json</code> and <code>gitlab.rb</code> (current ones are kept as <code>*.before-restore.<time></code>), restarts GitLab	Without the matching secrets, CI variables, runner tokens and 2FA can't be decrypted
3	Copies the backup into the container	<code>gitlab-backup</code> reads from <code>/var/opt/gitlab/backups</code>
4	Stops <code>puma</code> and <code>sidekiq</code>	Nothing may write to the database during the restore
5	<code>gitlab-backup restore</code>	Restores the database, repositories and uploads
6	Restarts, waits until ready, runs <code>gitlab:check</code> and <code>gitlab:doctor:secrets</code>	Confirms GitLab works and every secret can be decrypted

If the versions differ, start the backup's version first: in `docker-compose.yml` set `image: 'gitlab/gitlab-ce:<version>-ce.0'` (e.g. `17.4.0-ce.0`), run `docker compose up -d`, then restore.

Restoring onto a new VM (e.g. the old one was lost):

1. Deploy GitLab on the new VM (point `GITLAB_VM_IP` at it and re-run the workflow, or follow *Manual install* below).
2. Copy the backup folder to the new VM: `scp -r gitlab-backups <user>@<new-vm>:/tmp/`
3. `sudo /opt/gitra-platform/scripts/restore.sh /tmp/gitlab-backups`
4. Re-run the workflow so the runner re-registers if needed.

Server access (admin SSH)

Admin SSH (port 22) on the GitLab VM accepts **SSH keys only** — every deployment turns password login off (`scripts/harden_ssh.sh`). Root login over SSH is off too. GitHub Actions is unaffected: it uses its own key (`gitra-deploy`).

Add your own key (once, in [Azure Cloud Shell](#), Bash)

```
# 1. Create a key pair (skip if you already have one)
ssh-keygen -t ed25519 -N "" -C "gitra-admin" -f ~/.ssh/gitra_admin

# 2. Add the public key to the "gitra" user on the GitLab VM (through the Azure VM agent)
az vm user update -g gitra-rg -n gitra-gitlab-vm \
  -u gitra --ssh-key-value "$(cat ~/.ssh/gitra_admin.pub)"

# 3. Log in
ssh -i ~/.ssh/gitra_admin gitra@gitra-25ee91e6.eastus.cloudapp.azure.com
```

*Cloud Shell is **Bash**: paths use `/` (`~/.ssh/...`). The Windows form `$HOME\.ssh\...` fails there with **Permission denied**.*

Or from your own Windows computer (PowerShell)

```
# 1. Create a key pair (press Enter twice for no passphrase)
mkdir "$HOME\.ssh" -Force
ssh-keygen -t ed25519 -C "gitra-admin" -f "$HOME\.ssh\gitra_admin"
```

```
# 2. Show the public key and copy the whole line (starts with ssh-ed25519)
Get-Content "$HOME\.ssh\gitra_admin.pub"
```

Step 3. Add it to the VM — either in Cloud Shell: `az vm user update -g gitra-rg -n gitra-gitlab-vm -u gitra --ssh-key-value "<the copied line>"`, or in the portal: **gitra-gitlab-vm** → **Help** → **Reset password** → **Add SSH public key**, username **gitra**, paste the key → **Update**.

Step 4. Log in from PowerShell:

```
ssh -i "$HOME\.ssh\gitra_admin" gitra@gitra-25ee91e6.eastus.cloudapp.azure.com
```

Never share the private key (the file without `.pub`).

Without SSH

Azure portal → **gitra-gitlab-vm** → **Operations** → **Run command** → **RunShellScript** runs commands as root, no SSH needed.

Turning password login back on (not recommended)

Delete `/etc/ssh/sshd_config.d/00-gitra-hardening.conf` (via Run command) and run `systemctl reload sshd`. The next deployment turns it off again unless `harden_ssh.sh` is removed from `remote_deploy_gitlab.sh`.

Manual install (without GitHub Actions)

On a fresh Ubuntu VM:

```
git clone -b Testing https://github.com/Mohammed-Alghumayti/gitra-platform.git
cd gitra-platform && chmod +x scripts/*.sh
./scripts/setup.sh && newgrp docker
cp .env.example .env # set GITLAB_EXTERNAL_URL
./scripts/deploy.sh
./scripts/health_check.sh 900
./scripts/harden_gitlab.sh
```

For a runner on another VM: `./scripts/deploy_runner.sh`, then `./scripts/register_runner.sh <glrt-token> <gitlab_url>` (token from **Admin** → **CI/CD** → **Runners** → **New instance runner**).

Changing `GITLAB_EXTERNAL_URL` needs `docker compose down && docker compose up -d` — a restart does not re-read it.

Local checks (no Azure needed)

```
# Terraform
for d in terraform/gitlab terraform/runner; do (cd $d && terraform init -backend=false && terraform validate && terraform test); done

# Scripts and workflow
shellcheck scripts/*.sh scripts/ci/*.sh
actionlint .github/workflows/deploy.yml
docker compose config -q && docker compose -f docker-compose.runner.yaml config -q

# Demo app
cd sample-app && pip install -r requirements.txt && pytest -v
```

Part 3 — Troubleshooting & Change Log

What this covers

Every problem found while building and deploying the platform — what caused it and how it was fixed — plus a quick guide for problems you may hit later, and what's still worth improving.

Change log

v1.0 — Docker automation (Member 3)

GitLab CE in Docker Compose with persistent storage, plus `setup.sh`, `deploy.sh` and `health_check.sh`.

v2.0 — First fixes

#	Issue	Fix
1	<code>external_url</code> was <code>http://localhost</code> while the VM is reached at <code>20.55.88.3</code> — broken clone URLs and UI links	Set to the VM's address (later: its HTTPS DNS name)
2	<code>deploy.sh</code> set storage folders to <code>chmod 777</code> — anyone on the VM could read GitLab's secrets	Locked down (final form: problem 9 below)
3	No backup/restore path	Added <code>backup.sh</code> (backup + <code>gitlab-secrets.json</code> + <code>gitlab.rb</code>)
4	<code>gitlab/</code> (secrets, data) wasn't excluded from Git	Added <code>.gitignore</code>
5	Scripts contained leftover <code>cat << EOF</code> wrappers and didn't run	Cleaned up
6	<code>health_check.sh</code> checked only once, while GitLab takes minutes to boot	Polls until ready or timeout

v3.0 — Integration of the three workstreams (Testing branch)

#	Change
1	Merged Member 3's Docker work and Member 2's GitLab/CI work; one compose file instead of two
2	Member 2's demo app moved to <code>sample-app/</code> , with real pytest tests, a <code>/health</code> route, a non-root Dockerfile and a <code>deploy_staging</code> stage
3	Member 1's Azure infrastructure written in Terraform, with tests using a mocked Azure provider
4	Git SSH standardised on port 2224 (Member 2's file used 2222) to match the NSG

v3.1 — Security design

#	Risk	Solution
1	The runner mounts the Docker socket, so CI jobs could take over the GitLab server	Runner moved to its own VM
2	The site must be public for supervisors and students	HTTPS (Let's Encrypt), admin-approved sign-up, mandatory 2FA, 12-char passwords, no public projects, fail2ban, minimal NSG rules

v4.0 — Fully automatic deployment

GitHub Actions deploys everything on each push to **Testing**, using the team's existing GitLab VM and a Terraform-built runner VM. Secrets are stored in GitHub Secrets.

v4.1 → v4.8 — Problems found on the first real Azure deployments

Azure

#	Problem	Cause	Fix
1	exceeding approved Total Regional Cores quota	The subscription allows 4 vCPUs per region; the GitLab VM (Standard_D4s_v4) uses all 4 in East US	Runner VM placed in another region — quotas are per region
2	Terraform would have clashed with the existing VM	The GitLab VM lives in <code>gitra-rg</code> , the name Terraform also used	Runner resources renamed <code>gitra-runner-*</code> ; a test guards against reuse
3	Cannot modify extensions in the VM when the VM is not running	The VM was stopped (auto-shutdown)	The workflow starts the VM if it isn't running
4	Connection ... closed by remote host mid-deploy	The VM was shut down during the run	Dropped SSH connections are retried (all steps are idempotent)
5	SkuNotAvailable / Allocation failed	No capacity for the requested size in the region — common on student subscriptions	Try 7 small 2-vCPU sizes across 5 regions; first one Azure accepts wins
6	Every later size failed with already exists	A failed allocation left a half-created VM outside Terraform's state	Delete leftover VMs and unattached disks before each attempt

GitLab

#	Problem	Cause	Fix
7	Security settings failed with <code>database.yml</code> is a symlink that does not point to a valid file	<code>health_check.sh</code> accepted nginx's 301 redirect, which is returned before Rails has started	Check GitLab's own readiness endpoint (<code>/-/readiness</code>) inside the container
8	Container stayed unhealthy although GitLab served pages	Docker's built-in health check doesn't cope with the HTTPS setup	Same readiness endpoint, independent of HTTP vs HTTPS
9	GitLab didn't come back after a VM restart: <code>Permission denied - puma.rb</code> , sidekiq down, container restart loop	An earlier <code>chmod -R 700 / chown -R root</code> on the data folders locked GitLab's internal users (<code>git</code> , <code>gitlab-www</code> , ...) out of their own files. It only showed after a restart, when GitLab re-read its files	<code>deploy.sh</code> now locks only the parent <code>./gitlab</code> folder. The deploy detects when <code>git</code> can't read its files, reopens the folders (PostgreSQL's stays <code>700</code>), runs <code>update-permissions</code> and restarts GitLab so it resets exact permissions

Terraform

#	Problem	Cause	Fix
10	subnet ... was not found after moving the runner to another region	Replacing the resource group deleted the subnet, but a subnet has no region attribute, so Terraform assumed it still existed	<code>replace_triggered_by</code> rebuilds the subnet with its VNet, and the NIC–NSG link with the NIC

Result

The full run passed end to end: GitLab over HTTPS with all security settings, root password set, runner VM created (`Standard_B2s_v2`, West US 2) and registered, and the demo pipeline `test` → `build` → `deploy_staging` passed.

v4.9 — Platform in Terraform, SSH keys only, restore

#	Change	Why
1	Terraform split in two: <code>terraform/gitlab</code> adopts the existing GitLab platform (RG, VNet, subnet, NSG, public IP, NIC, VM) with <code>terraform import;</code> <code>terraform/runner</code> keeps the runner	The spec asks for the platform itself in Terraform. Import adopts it without recreating the VM; the workflow refuses any plan that would delete or change it
2	Admin SSH on the GitLab VM: password login off (<code>harden_ssh.sh</code>)	The VM was created with password login on; bots on the internet guess passwords on port 22 all day. Keys can't be guessed. Done inside the VM, not in Terraform, because changing it there would rebuild the VM
3	<code>restore.sh</code> + restore steps in the docs	A backup is only useful if it can be restored. Tested locally: project deleted → restored with its repository
4	Two-VM design justified against the spec's single-node model	See <i>Architecture</i>

#	Problem	Cause	Fix
11	First platform plan stopped with <code>Not applied – the plan would modify ... azurerm_linux_virtual_machine.gitlab</code>	The VM has hibernation and Ultra SSD explicitly <code>false</code> ; the config didn't say so, so Terraform planned to clear them. The safety check worked: nothing was changed	<code>additional_capabilities</code> set to match Azure
12	Every run tried to resize the runner VM (<code>Standard_B2als_v2</code> → <code>Standard_B2s</code>), failing with <code>SkuNotAvailable</code>	The size list always started with <code>Standard_B2s</code> , whatever size the VM already had	An existing VM's own size and region are tried first — a working VM is never resized
13	<code>ssh: connect to host ... port 22: Connection timed out</code> waiting for the runner	The runner VM was stopped (deallocated)	The workflow starts the runner VM if it isn't running, and fails with a clear message if it stays unreachable
14	Starting the stopped runner VM failed: <code>AllocationFailed</code>	A deallocated VM gives up its hardware; West US 2 had no capacity left to start it again	Starting the VM is part of each attempt in the size/region loop: if it can't start, the next size is tried (resized while stopped), then the next region (Terraform rebuilds the disposable runner VM, which

#	Problem	Cause	Fix
			re-registers itself)

Lessons learned

- **Reproduce before fixing.** The permission bug was recreated on a local GitLab (same error, same restart loop) and the repair was proven there before it touched the real VM.
- **Don't trust a single signal.** An HTTP answer or Docker's health status isn't proof the application is ready.
- **Make every step idempotent,** so a failed deployment can simply be re-run.
- **Least privilege must fit the software.** Locking everything down broke GitLab; lock the outer folder and let the application manage its own permissions.
- **Cloud capacity isn't guaranteed,** especially on student subscriptions — design for fallbacks.

Troubleshooting guide

Where to look first

- **GitHub** → **Actions** → the failed run → the red step. Its log ends with the actual error.
- **GitLab logs** (Azure portal → **gitra-gitlab-vm** → **Operations** → **Run command** → **RunShellScript**, no SSH needed):

```
docker logs gitlab_server --tail 100
docker exec gitlab_server gitlab-ctl status
```

Common problems

Symptom	Likely cause	What to do
Site shows 502	GitLab is still starting	Wait 3–5 minutes
Site doesn't open at all	GitLab VM is stopped	Start it in the portal, or re-run the workflow
Browser says the certificate isn't secure	Let's Encrypt not issued yet	Check port 80 is open; <code>docker logs gitlab_server grep -i letsencrypt</code>
Workflow: Deployment skipped – missing secrets	Secrets not added	Add the 3 secrets (<i>Deployment</i>)
Workflow: No public IP ... in this subscription	Wrong GITLAB_VM_IP	Set the GITLAB_VM_IP variable
Workflow: No runner VM size had capacity	Azure has no capacity in the tried regions	Set RUNNER_LOCATION to another region your subscription allows
Workflow: Runner VM (...) not reachable over SSH	Runner VM still booting, or its NSG/SSH is broken	Re-run; if it persists, check gitra-runner-vm → Boot diagnostics in the portal
Runner shows offline in GitLab	Runner VM stopped, or token lost	Start the VM; if still offline, re-run the workflow (it re-registers when needed)
Pipeline stuck in pending	No online runner	See the previous row

Symptom	Likely cause	What to do
Permission denied - puma.rb in GitLab logs	File permissions broken	Re-run the workflow — it repairs them. Manually: <code>docker exec gitlab_server update-permissions && docker restart gitlab_server</code>
git push rejects the password	2FA is on	Use a personal access token (Edit profile → Access tokens , scope <code>write_repository</code>) as the password
New user can't log in	Account pending	Admin → Users → Pending approval → Approve
ssh gitra@... says Permission denied (publickey)	Admin SSH accepts keys only	Add your key (<i>Deployment</i>), or use Run command in the portal
restore.sh: Versions differ	Backup made by another GitLab version	Start that version first (<i>Deployment</i>)
Deploy fails at harden_ssh.sh: No SSH keys found	No user on the VM has a key — turning passwords off would lock everyone out	Check step 1 added the deploy user's key, then re-run

Still worth improving

- ☐ Pin the GitLab and runner image versions instead of `latest`, so upgrades are deliberate.
- ☐ Copy encrypted backups off the VM (e.g. Azure Blob Storage) on a schedule.
- ☐ Serve the staging app over HTTPS.
- ☐ Copy the workflow to `main` so the **Run workflow** button is available.